

6

Binary decision diagrams

6.1 Representing boolean functions

Boolean functions are an important descriptive formalism for many hardware and software systems, such as synchronous and asynchronous circuits, reactive systems and finite-state programs. Representing those systems in a computer in order to reason about them requires an efficient representation for boolean functions. We look at such a representation in this chapter and describe in detail how the systems discussed in Chapter 3 can be verified using the representation.

Definition 6.1 A boolean variable x is a variable ranging over the values 0 and 1. We write $x_1, x_2, \dots$ and $x, y, z, \dots$ to denote boolean variables. We define the following functions on the set $\{0, 1\}$:

- $\overline{0} \stackrel{\text{def}}{=} 1$ and $\overline{1} \stackrel{\text{def}}{=} 0$;
- $x \cdot y \stackrel{\text{def}}{=} 1$ if x and y have value 1; otherwise $x \cdot y \stackrel{\text{def}}{=} 0$;
- $x + y \stackrel{\text{def}}{=} 0$ if x and y have value 0; otherwise $x + y \stackrel{\text{def}}{=} 1$;
- $x \oplus y \stackrel{\text{def}}{=} 1$ if exactly one of x and y equals 1.

A boolean function f of n arguments is a function from $\{0, 1\}^n$ to $\{0, 1\}$. We write $f(x_1, x_2, \dots, x_n)$, or $f(V)$, to indicate that a syntactic representation of f depends on the boolean variables in V only.

Note that $\cdot$, $+$ and $\oplus$ are boolean functions with two arguments, whereas $\overline{}$ is a boolean function that takes one argument. The binary functions $\cdot$, $+$ and $\oplus$ are written in infix notation instead of prefix; i.e. we write $x + y$ instead of $+(x, y)$, etc.

Example 6.2 In terms of the four functions above, we can define other boolean functions such as

- (1) $f(x, y) \stackrel{\text{def}}{=} x \cdot (y + \bar{x})$
- (2) $g(x, y) \stackrel{\text{def}}{=} x \cdot y + (1 \oplus \bar{x})$
- (3) $h(x, y, z) \stackrel{\text{def}}{=} x + y \cdot (x \oplus \bar{y})$
- (4) $k() \stackrel{\text{def}}{=} 1 \oplus (0 \cdot \bar{1})$.

6.1.1 Propositional formulas and truth tables

Truth tables and *propositional formulas* are two different representations of boolean functions. In propositional formulas, $\wedge$ denotes $\cdot$, $\vee$ denotes $+$, $\neg$ denotes $\bar{}$ and $\top$ and $\perp$ denote 1 and 0, respectively.

Boolean functions are represented by truth tables in the obvious way; for example, the function $f(x, y) \stackrel{\text{def}}{=} \overline{x + y}$ is represented by the truth table on the left:

x	y	$f(x, y)$	p	q	ϕ
1	1	0	T	T	F
0	1	0	F	T	F
1	0	0	T	F	F
0	0	1	F	F	T

On the right, we show the same truth table using the notation of Chapter 1; a formula having this truth table is $\neg(p \vee q)$. In this chapter, we may mix these two notational systems of boolean formulas and formulas of propositional logic whenever it is convenient. You should be able to translate expressions easily from one notation to the other and vice versa.

As representations of boolean functions, propositional formulas and truth tables have different advantages and disadvantages. Truth tables are very space-inefficient: if one wanted to model the functionality of a sequential circuit by a boolean function of 100 variables (a small chip component would easily require this many variables), then the truth table would require 2^{100} (which is more than 10^{30}) lines. Alas, there is not enough storage space (whether paper or particle) in the universe to record the information of 2^{100} different bit vectors of length 100. Although they are space inefficient, operations on truth tables are simple. Once you have computed a truth table, it is easy to see whether the boolean function represented is satisfiable: you just look to see if there is a 1 in the last column of the table.

Comparing whether two truth tables represent the same boolean function also seems easy: assuming the two tables are presented with the same order

of valuations, we simply check that they are identical. Although these operations seem simple, however, they are computationally intractable because of the fact that the number of lines in the truth table is exponential in the number of variables. Checking satisfiability of a function with n atoms requires of the order of 2^n operations if the function is represented as a truth table. We conclude that checking satisfiability and equivalence is highly inefficient with the truth-table representation.

Representation of boolean functions by propositional formulas is slightly better. Propositional formulas often provide a wonderfully compact and efficient presentation of boolean functions. A formula with 100 variables might only be about 200–300 characters long. However, deciding whether an arbitrary propositional formula is satisfiable is a famous problem in computer science: no efficient algorithms for this task are known, and it is strongly suspected that there aren't any. Similarly, deciding whether two arbitrary propositional formulas f and g denote the same boolean function is suspected to be exponentially expensive.

It is straightforward to see how to perform the boolean operations $\cdot$, $+$, $\oplus$ and $\neg$ on these two representations. In the case of truth tables, they involve applying the operation to each line; for example, given truth tables for f and g over the same set of variables (and in the same order), the truth table for $f \oplus g$ is obtained by applying $\oplus$ to the truth value of f and g in each line. If f and g do not have the same set of arguments, it is easy to pad them out by adding further arguments. In the case of representation by propositional formulas, the operations $\cdot$, $\oplus$, etc., are simply syntactic manipulations. For example, given formulas ϕ and ψ representing the functions f and g , the formulas representing $f \cdot g$ and $f \oplus g$ are, respectively, $\phi \wedge \psi$ and $(\phi \wedge \neg\psi) \vee (\neg\phi \wedge \psi)$.

We could also consider representing boolean functions by various subclasses of propositional formulas, such as conjunctive and disjunctive normal forms. In the case of **disjunctive normal form** (DNF, in which a formula is a disjunction of conjunctions of literals), **the representation is sometimes compact, but in the worst cases it can be very lengthy**. Checking satisfiability is a straightforward operation, however, because it is sufficient to find a disjunct which does not have two complementary literals. **Unfortunately, there is not a similar way of checking validity**. Performing $+$ on two formulas in DNF simply involves inserting $\vee$ between them. Performing $\cdot$ is more complicated; we cannot simply insert $\wedge$ between the two formulas, because the result will not in general be in DNF, so we have to perform lengthy applications of the distributivity rule $\phi \wedge (\psi_1 \vee \psi_2) \equiv (\phi \wedge \psi_1) \vee (\phi \wedge \psi_2)$. **Computing the negation of a DNF formula is also expensive**. The DNF formula ϕ may be

Representation of boolean functions	compact?	test for		boolean operations		
		satisfy	validity	$\cdot$	$+$	$-$
Prop. formulas	often	hard	hard	easy	easy	easy
Formulas in DNF	sometimes	easy	hard	hard	easy	hard
Formulas in CNF	sometimes	hard	easy	easy	hard	hard
Ordered truth tables	never	hard	hard	hard	hard	hard
Reduced OBDDs	often	easy	easy	medium	medium	easy

Figure 6.1. Comparing efficiency of five representations of boolean formulas.

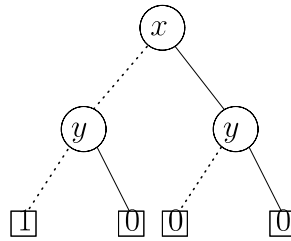


Figure 6.2. An example of a binary decision tree.

quite short, whereas the length of the disjunctive normal form of $\neg\phi$ can be exponential in the length of ϕ .

The situation for representation in conjunctive normal form is the dual. A summary of these remarks is contained in Figure 6.1 (for now, please ignore the last row).

6.1.2 Binary decision diagrams

Binary decision diagrams (BDDs) are another way of representing boolean functions. A certain class of such diagrams will provide the implementational framework for our symbolic model-checking algorithm. Binary decision diagrams were first considered in a simpler form called *binary decision trees*. These are trees whose non-terminal nodes are labelled with boolean variables $x, y, z, \dots$ and whose terminal nodes are labelled with either 0 or 1. Each non-terminal node has two edges, one dashed line and one solid line. In Figure 6.2 you can see such a binary decision tree with two layers of variables x and y .

Definition 6.3 Let T be a finite binary decision tree. Then T determines a unique boolean function of the variables in non-terminal nodes, in the following way. Given an assignment of 0s and 1s to the boolean variables

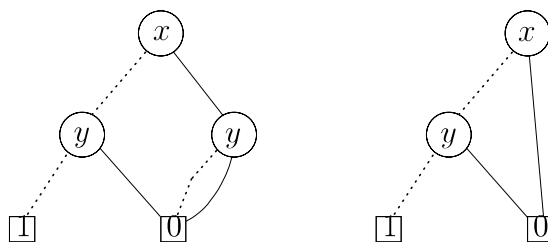


Figure 6.3. (a) Sharing the terminal nodes of the binary decision tree in Figure 6.2; (b) further optimisation by removing a redundant decision point.

occurring in T , we start at the root of T and take the dashed line whenever the value of the variable at the current node is 0; otherwise, we travel along the solid line. The function value is the value of the terminal node we reach.

For example, the binary decision tree of Figure 6.2 represents a boolean function $f(x, y)$. To find $f(0, 1)$, start at the root of the tree. Since the value of x is 0 we follow the dashed line out of the node labelled x and arrive at the leftmost node labelled y . Since y 's value is 1, we follow the solid line out of that y -node and arrive at the leftmost terminal node labelled 0. Thus, $f(0, 1)$ equals 0. In computing $f(0, 0)$, we similarly travel down the tree, but now following two dashed lines to obtain 1 as a result. You can see that the two other possibilities result in reaching the remaining two terminal nodes labelled 0. Thus, this binary decision tree computes the function $f(x, y) \stackrel{\text{def}}{=} \overline{x + y}$.

Binary decision trees are quite close to the representation of boolean functions as truth tables as far as their sizes are concerned. If the root of a binary decision tree is an x -node then it has two subtrees (one for the value of x being 0 and another one for x having value 1). So if f depends on n boolean variables, the corresponding binary decision tree will have at least $2^{n+1} - 1$ nodes (see exercise 5 on page 399). Since f 's truth table has 2^n lines, we see that decision trees as such are not a more compact representation of boolean functions. However, binary decision trees often contain some redundancy which we can exploit.

Since 0 and 1 are the only terminal nodes of binary decision trees, we can optimise the representation by having pointers to just one copy of 0 and one copy of 1. For example, the binary decision tree in Figure 6.2 can be optimised in this way and the resulting structure is depicted in Figure 6.3(a). Note that we saved storage space for two redundant terminal 0-nodes, but that we still have as many edges (pointers) as before.

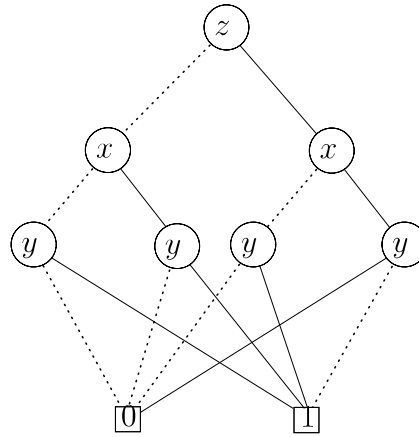


Figure 6.4. A BDD with duplicated subBDDs.

A second optimisation we can do is to remove unnecessary decision points in the tree. In Figure 6.3(a), the right-hand y is unnecessary, because we go to the same place whether it is 0 or 1. Therefore the structure could be further reduced, to the one shown on the right, (b).

All these structures are examples of *binary decision diagrams* (BDDs). They are more general than binary decision trees; the sharing of the leaves means they are not trees. As a third optimisation, we also allow subBDDs to be shared. A subBDD is the part of a BDD occurring below a given node. For example, in the BDD of Figure 6.4, the two inner y -nodes perform the same role, because the subBDDs below them have the same structure. Therefore, one of them could be removed, resulting in the BDD in Figure 6.5(a). Indeed, the left-most y -node could also be merged with the middle one; then the x -node above both of them would become redundant. Removing it would result in the BDD on the right of Figure 6.5.

To summarise, we encountered three different ways of reducing a BDD to a more compact form:

C1. Removal of duplicate terminals. If a BDD contains more than one terminal 0-node, then we redirect all edges which point to such a 0-node to just one of them. We proceed in the same way with terminal nodes labelled with 1.

C2. Removal of redundant tests. If both outgoing edges of a node n point to the same node m , then we eliminate that node n , sending all its incoming edges to m .

C3. Removal of duplicate non-terminals. If two distinct nodes n and m in the BDD are the roots of structurally identical subBDDs, then we

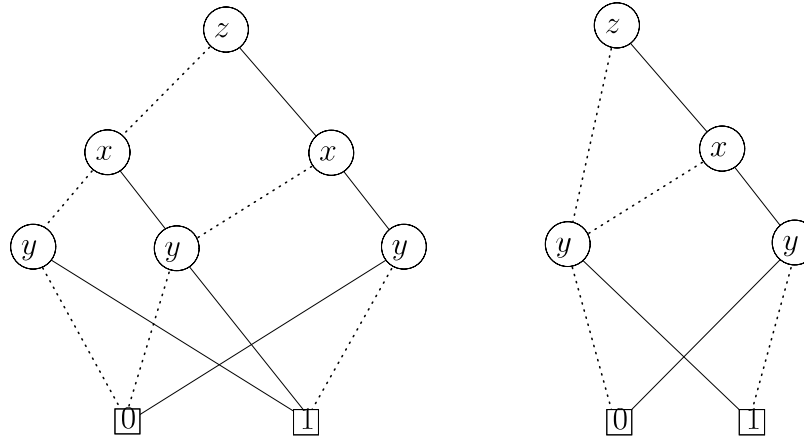


Figure 6.5. The BDD of Figure 6.4: (a) after removal of one of the duplicate y -nodes; (b) after removal of another duplicate y -node and then a redundant x -decision point.

eliminate one of them, say m , and redirect all its incoming edges to the other one.

Note that C1 is a special case of C3. In order to define BDDs precisely, we need a few auxiliary notions.

Definition 6.4 A directed graph is a set G and a binary relation $\rightarrow$ on G : $\rightarrow \subseteq G \times G$. A cycle in a directed graph is a finite path in that graph that begins and ends at the same node, i.e. a path of the form $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n \rightarrow v_1$. A directed acyclic graph (dag) is a directed graph that does not have any cycles. A node of a dag is initial if there are no edges pointing to that node. A node is called terminal if there are no edges out of that node.

The directed graph in Figure 3.3 on page 179 has cycles, for example the cycle $s_0 \rightarrow s_1 \rightarrow s_0$, and is not a dag. If we interpret the links in BDDs (whether solid or dashed) as always going in a downwards direction, then the BDDs of this chapter are also directed graphs. They are also acyclic and have a unique initial node. The optimisations C1–C3 preserve the property of being a dag; and fully reduced BDDs have precisely two terminal nodes. We now formally define BDDs as certain kinds of dags:

Definition 6.5 A binary decision diagram (BDD) is a finite dag with a unique initial node, where all terminal nodes are labelled with 0 or 1 and all non-terminal nodes are labelled with a boolean variable. Each



Figure 6.6. The BDDs (a) B_0 , representing the constant 0 boolean function; similarly, the BDD B_1 has only one node 1 and represents the constant 1 boolean function; and (b) B_x , representing the boolean variable x .

non-terminal node has exactly two edges from that node to others: one labelled 0 and one labelled 1 (we represent them as a dashed line and a solid line, respectively).

A BDD is said to be reduced if none of the optimisations C1–C3 can be applied (i.e. no more reductions are possible).

All the decision structures we have seen in this chapter (Figures 6.2–6.5) are BDDs, as are the constant functions B_0 and B_1 , and the function B_x from Figure 6.6. If B is a BDD where $V = \{x_1, x_2, \dots, x_n\}$ is the set of labels of non-terminal nodes, then B determines a boolean function $f(V)$ in the same way as binary decision trees (see Definition 6.3): given an assignment of 0s and 1s to the variables in V , we compute the value of f by starting with the unique initial node. If its variable has value 0, we follow the dashed line; otherwise we take the solid line. We continue for each node until we reach a terminal node. Since the BDD is finite by definition, we eventually reach a terminal node which is labelled with 0 or 1. That label is the result of f for that particular assignment of truth values.

The definition of a BDD does not prohibit that a boolean variable occur more than once on a path in the dag. For example, consider the BDD in Figure 6.7.

Such a representation is wasteful, however. The solid link from the left-most x to the 1-terminal is never taken, for example, because one can only get to that x -node when x has value 0.

Thanks to the reductions C1–C3, BDDs can often be quite compact representations of boolean functions. Let us consider how to check satisfiability and perform the boolean operations on functions represented as BDDs. A BDD represents a satisfiable function if a 1-terminal node is reachable from the root along a *consistent path* in a BDD which represents it. A consistent path is one which, for every variable, has only dashed lines or only solid lines leaving nodes labelled by that variable. (In other words, we cannot assign

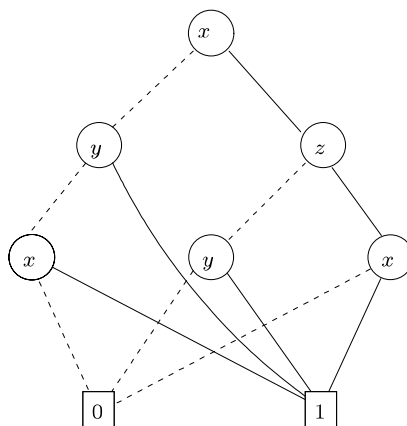


Figure 6.7. A BDD where some boolean variables occur more than once on an evaluation path.

a variable the values 0 and 1 simultaneously.) Checking validity is similar, but we check that no 0-terminal is reachable by a consistent path.

The operations $\cdot$ and $+$ can be performed by ‘surgery’ on the component BDDs. Given BDDs B_f and B_g representing boolean functions f and g , a BDD representing $f \cdot g$ can be obtained by taking the BDD f and replacing all its 1-terminals by B_g . To see why this is so, consider how to get to a 1-terminal in the resulting BDD. You have to satisfy the requirements for getting to a 1 imposed by both of the BDDs. Similarly, a BDD for $f + g$ can be obtained by replacing all 0 terminals of B_f by B_g . Note that these operations are likely to generate BDDs with multiple occurrences of variables along a path. Later, in Section 6.2, we will see definitions of $+$ and $\cdot$ on BDDs that don’t have this undesirable effect.

The complementation operation $\bar{}$ is also possible: a BDD representing $\bar{f}$ can be obtained by replacing all 0-terminals in B_f by 1-terminals and vice versa. Figure 6.8 shows the complement of the BDD in Figure 6.2.

6.1.3 Ordered BDDs

We have seen that the representation of boolean functions by BDDs is often compact, thanks to the sharing of information afforded by the reductions C1–C3. However, BDDs with multiple occurrences of a boolean variable along a path seem rather inefficient. Moreover, there seems no easy way to test for equivalence of BDDs. For example, the BDDs of Figures 6.7 and 6.9 represent the same boolean function (the reader should check this). Neither of them can be optimised further by applying the rules C1–C3. However,

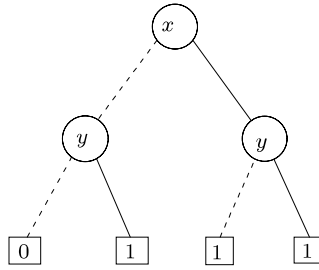


Figure 6.8. The complement of the BDD in Figure 6.2.

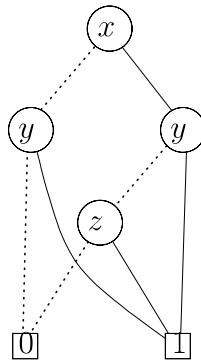


Figure 6.9. A BDD representing the same function as the BDD of Figure 6.7, but having the variable ordering $[x, y, z]$.

testing whether they denote the same boolean function seems to involve as much computational effort as computing the entire truth table for $f(x, y, z)$.

We can improve matters by imposing an ordering on the variables occurring along any path. We then adhere to that same ordering for all the BDDs we manipulate.

Definition 6.6 Let $[x_1, \dots, x_n]$ be an ordered list of variables without duplications and let B be a BDD all of whose variables occur somewhere in the list. We say that B has the ordering $[x_1, \dots, x_n]$ if all variable labels of B occur in that list and, for every occurrence of x_i followed by x_j along any path in B , we have $i < j$.

An ordered BDD (OBDD) is a BDD which has an ordering for some list of variables.

Note that the BDDs of Figures 6.3(a,b) and 6.9 are ordered (with ordering $[x, y]$). We don't insist that every variable in the list is used in the paths. Thus, the OBDDs of Figures 6.3 and 6.9 have the ordering $[x, y, z]$ and so

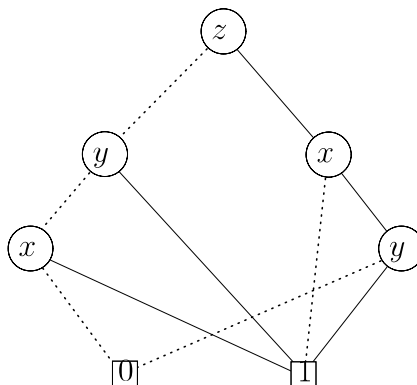


Figure 6.10. A BDD which does not have an ordering of variables.

does any list having x , y and z in it in that order, such as $[u, x, y, v, z, w]$ and $[x, u, y, z]$. Even the BDDs B_0 and B_1 in Figure 6.6 are OBDDs, a suitable ordering list being the empty list (there are no variables), or indeed *any* list. The BDD B_x of Figure 6.6(b) is also an OBDD, with any list containing x as its ordering.

The BDD of Figure 6.7 is not ordered. To see why this is so, consider the path taken if the values of x and y are 0. We begin with the root, an x -node, and reach a y -node and then an x -node again. Thus, no matter what list arrangement we choose (remembering that no double occurrences are allowed), this path violates the ordering condition. Another example of a BDD that is not ordered can be seen in Figure 6.10. In that case, we cannot find an order since the path for $(x, y, z) \Rightarrow (0, 0, 0)$ – meaning that x , y and z are assigned 0 – shows that y needs to occur before x in such a list, whereas the path for $(x, y, z) \Rightarrow (1, 1, 1)$ demands that x be before y .

It follows from the definition of OBDDs that one cannot have multiple occurrences of any variable along a path.

When operations are performed on two OBDDs, we usually require that they have *compatible variable orderings*. The orderings of B_1 and B_2 are said to be compatible if there are no variables x and y such that x comes before y in the ordering of B_1 and y comes before x in the ordering of B_2 . This commitment to an ordering gives us a unique representation of boolean functions as OBDDs. For example, the BDDs in Figures 6.8 and 6.9 have compatible variable orderings.

Theorem 6.7 The reduced OBDD representing a given function f is unique. That is to say, let B and B' be two reduced OBDDs with

compatible variable orderings. If B and B represent the same boolean function, then they have identical structure.

In other words, with OBDDs we cannot get a situation like the one encountered earlier, in which we have two distinct reduced BDDs which represent the same function, provided that the orderings are compatible. It follows that checking equivalence of OBDDs is immediate. Checking whether two OBDDs (having compatible orderings) represent the same function is simply a matter of checking whether they have the same structure¹.

A useful consequence of the theorem above is that, if we apply the reductions C1–C3 to an OBDD until no further reductions are possible, then we are guaranteed that the result is always the same reduced OBDD. The order in which we applied the reductions does not matter. We therefore say that OBDDs have a *canonical form*, namely their unique reduced OBDD. Most other representations (conjunctive normal forms, etc.) do not have canonical forms.

The algorithms for $\cdot$ and $+$ for BDDs, presented in Section 6.1.2, won't work for OBDDs as they may introduce multiple occurrences of the same variable on a path. We will soon develop more sophisticated algorithms for these operations on OBDDs, which exploit the compatible ordering of variables in paths.

OBDDs allow compact representations of certain classes of boolean functions which only have exponential representations in other systems, such as truth tables and conjunctive normal forms. As an example consider the *even parity function* $f_{\text{even}}(x_1, x_2, \dots, x_n)$ which is defined to be 1 if there is an even number of variables x_i with value 1; otherwise, it is defined to be 0. Its representation as an OBDD requires only $2n + 1$ nodes. Its OBDD for $n = 4$ and the ordering $[x_1, x_2, x_3, x_4]$ can be found in Figure 6.11.

The impact of the chosen variable ordering The size of the OBDD representing the parity functions is independent of the chosen variable ordering. This is because the parity functions are themselves independent of the order of variables: swapping the values of any two variables does not change the value of the function; such functions are called symmetric.

However, in general the chosen variable ordering makes a significant difference to the size of the OBDD representing a given function. Consider the boolean function $(x_1 + x_2) \cdot (x_3 + x_4) \cdots (x_{2n-1} + x_{2n})$; it corresponds to a propositional formula in conjunctive normal form. If we choose the

¹ In an implementation this will amount to checking whether two pointers are equal.

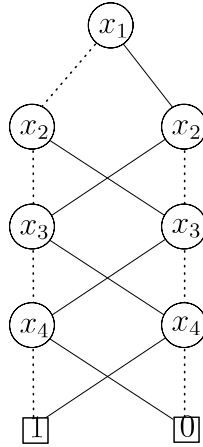


Figure 6.11. An OBDD for the even parity function for four bits.

‘natural’ ordering $[x_1, x_2, x_3, x_4, \dots]$, then we can represent this function as an OBDD with $2n + 2$ nodes. Figure 6.12 shows the resulting OBDD for $n = 3$. Unfortunately, if we choose instead the ordering

$$[x_1, x_3, \dots, x_{2n-1}, x_2, x_4, \dots, x_{2n}]$$

the resulting OBDD requires 2^{n+1} nodes; the OBDD for $n = 3$ can be seen in Figure 6.13.

The sensitivity of the size of an OBDD to the particular variable ordering is a price we pay for all the advantages that OBDDs have over BDDs. Although finding the optimal ordering is itself a computationally expensive problem, there are good heuristics which will usually produce a fairly good ordering. Later on we return to this issue in discussions of applications.

The importance of canonical representation The importance of having a canonical form for OBDDs in conjunction with an efficient test for deciding whether two reduced OBDDs are isomorphic cannot be overestimated. It allows us to perform the following tests:

Absence of redundant variables. If the value of the boolean function $f(x_1, x_2, \dots, x_n)$ does not depend on the value of x_i , then any reduced OBDD which represents f does not contain any x_i -node.

Test for semantic equivalence. If two functions $f(x_1, x_2, \dots, x_n)$ and $g(x_1, x_2, \dots, x_n)$ are represented by OBDDs B_f , respectively B_g , with a compatible ordering of variables, then we can efficiently decide whether f and g are semantically equivalent. We reduce B_f and B_g (if necessary); f

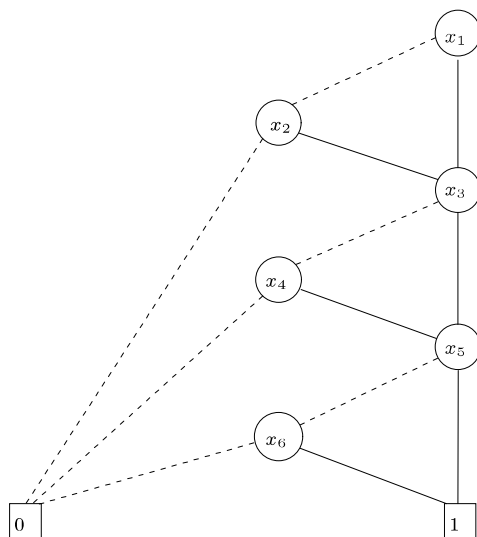


Figure 6.12. The OBDD for $(x_1 + x_2) \cdot (x_3 + x_4) \cdot (x_5 + x_6)$ with variable ordering $[x_1, x_2, x_3, x_4, x_5, x_6]$.

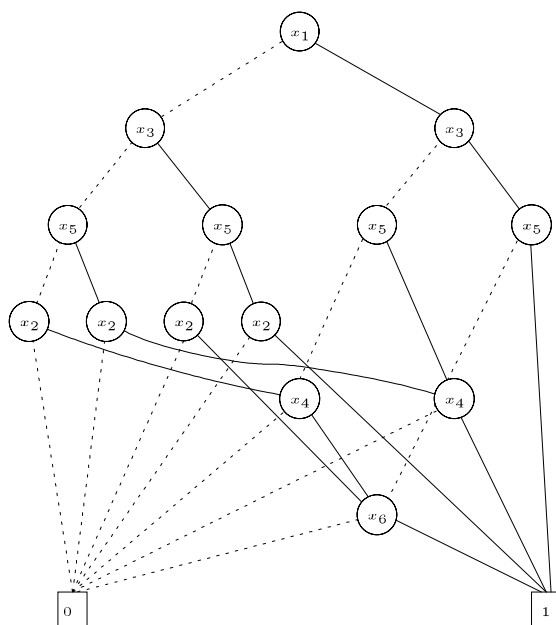


Figure 6.13. Changing the ordering may have dramatic effects on the size of an OBDD: the OBDD for $(x_1 + x_2) \cdot (x_3 + x_4) \cdot (x_5 + x_6)$ with variable ordering $[x_1, x_3, x_5, x_2, x_4, x_6]$.

and g denote the same boolean functions if, and only if, the reduced OBDDs have identical structure.

Test for validity. We can test a function $f(x_1, x_2, \dots, x_n)$ for validity (i.e. f always computes 1) in the following way. Compute a reduced OBDD for f . Then f is valid if, and only if, its reduced OBDD is B_1 .

Test for implication. We can test whether $f(x_1, x_2, \dots, x_n)$ implies $g(x_1, x_2, \dots, x_n)$ (i.e. whenever f computes 1, then so does g) by computing the reduced OBDD for $f \cdot \bar{g}$. This is B_0 iff the implication holds.

Test for satisfiability. We can test a function $f(x_1, x_2, \dots, x_n)$ for satisfiability (f computes 1 for at least one assignment of 0 and 1 values to its variables). The function f is satisfiable iff its reduced OBDD is not B_0 .

6.2 Algorithms for reduced OBDDs

6.2.1 The algorithm `reduce`

The reductions C1–C3 are at the core of any serious use of OBDDs, for whenever we construct a BDD we will want to convert it to its reduced form. In this section, we describe an algorithm `reduce` which does this efficiently for ordered BDDs.

If the ordering of B is $[x_1, x_2, \dots, x_l]$, then B has at most $l + 1$ layers. The algorithm `reduce` now traverses B layer by layer in a bottom-up fashion, beginning with the terminal nodes. In traversing B , it assigns an integer label $\text{id}(n)$ to each node n of B , in such a way that the subOBDDs with root nodes n and m denote the same boolean function if, and only if, $\text{id}(n)$ equals $\text{id}(m)$.

Since `reduce` starts with the layer of terminal nodes, it assigns the first label (say #0) to the first 0-node it encounters. All other terminal 0-nodes denote the same function as the first 0-node and therefore get the same label (compare with reduction C1). Similarly, the 1-nodes all get the next label, say #1.

Now let us inductively assume that `reduce` has already assigned integer labels to all nodes of a layer $> i$ (i.e. all terminal nodes and x_j -nodes with $j > i$). We describe how nodes of layer i (i.e. x_i -nodes) are being handled.

Definition 6.8 Given a non-terminal node n in a BDD, we define $\text{lo}(n)$ to be the node pointed to via the dashed line from n . Dually, $\text{hi}(n)$ is the node pointed to via the solid line from n .

Let us describe how the labelling is done. Given an x_i -node n , there are three ways in which it may get its label:

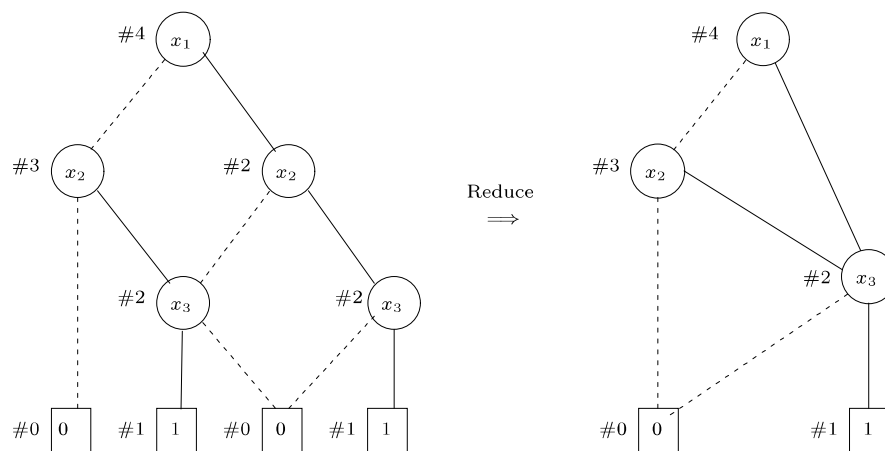


Figure 6.14. An example execution of the algorithm `reduce`.

- If the label $\text{id}(\text{lo}(n))$ is the same as $\text{id}(\text{hi}(n))$, then we set $\text{id}(n)$ to be that label. That is because the boolean function represented at n is the same function as the one represented at $\text{lo}(n)$ and $\text{hi}(n)$. In other words, node n performs a redundant test and can be eliminated by reduction C2.
- If there is another node m such that n and m have the same variable x_i , and $\text{id}(\text{lo}(n)) = \text{id}(\text{lo}(m))$ and $\text{id}(\text{hi}(n)) = \text{id}(\text{hi}(m))$, then we set $\text{id}(n)$ to be $\text{id}(m)$. This is because the nodes n and m compute the same boolean function (compare with reduction C3).
- Otherwise, we set $\text{id}(n)$ to the next unused integer label.

Note that only the last case creates a new label. Consider the OBDD in left side of Figure 6.14; each node has an integer label obtained in the manner just described. The algorithm `reduce` then finishes by redirecting edges bottom-up as outlined in C1–C3. The resulting reduced OBDD is in right of Figure 6.14. Since there are efficient bottom-up traversal algorithms for dags, `reduce` is an efficient operation in the number of nodes of an OBDD.

6.2.2 The algorithm `apply`

Another procedure at the heart of OBDDs is the algorithm `apply`. It is used to implement operations on boolean functions such as $+$, $\cdot$, $\oplus$ and complementation (via $f \oplus 1$). Given OBDDs B_f and B_g for boolean formulas f and g , the call `apply` (op, B_f, B_g) computes the reduced OBDD of the boolean formula $f \text{ op } g$, where op denotes any function from $\{0, 1\} \times \{0, 1\}$ to $\{0, 1\}$.

The intuition behind the **apply** algorithm is fairly simple. The algorithm operates recursively on the structure of the two OBDDs:

1. let v be the variable highest in the ordering (=leftmost in the list) which occurs in B_f or B_g .
2. split the problem into two subproblems for v being 0 and v being 1 and solve recursively;
3. at the leaves, apply the boolean operation op directly.

The result will usually have to be reduced to make it into an OBDD. Some reduction can be done ‘on the fly’ in step 2, by avoiding the creation of a new node if both branches are equal (in which case return the common result), or if an equivalent node already exists (in which case, use it).

Let us make all this more precise and detailed.

Definition 6.9 Let f be a boolean formula and x a variable.

1. We denote by $f[0/x]$ the boolean formula obtained by replacing all occurrences of x in f by 0. The formula $f[1/x]$ is defined similarly. The expressions $f[0/x]$ and $f[1/x]$ are called restrictions of f .
2. We say that two boolean formulas f and g are semantically equivalent if they represent the same boolean function (with respect to the boolean variables that they depend upon). In that case, we write $f \equiv g$.

For example, if $f(x, y) \stackrel{\text{def}}{=} x \cdot (y + \bar{x})$, then $f[0/x](x, y)$ equals $0 \cdot (y + \bar{0})$, which is semantically equivalent to 0. Similarly, $f[1/y](x, y)$ is $x \cdot (1 + \bar{x})$, which is semantically equivalent to x .

Restrictions allow us to perform recursion on boolean formulas, by decomposing boolean formulas into simpler ones. For example, if x is a variable in f , then f is equivalent to $\bar{x} \cdot f[0/x] + x \cdot f[1/x]$. To see this, consider the case $x = 0$; the expression computes to $f[0/x]$. When $x = 1$ it yields $f[1/x]$. This observation is known as the *Shannon expansion*, although it can already be found in G. Boole’s book ‘*The Laws of Thought*’ from 1854.

Lemma 6.10 (Shannon expansion) For all boolean formulas f and all boolean variables x (even those not occurring in f) we have

$$f \equiv \bar{x} \cdot f[0/x] + x \cdot f[1/x]. \quad (6.1)$$

The function **apply** is based on the Shannon expansion for $f \text{ op } g$:

$$f \text{ op } g = \bar{x}_i \cdot (f[0/x_i] \text{ op } g[0/x_i]) + x_i \cdot (f[1/x_i] \text{ op } g[1/x_i]). \quad (6.2)$$

This is used as a control structure of **apply** which proceeds from the roots

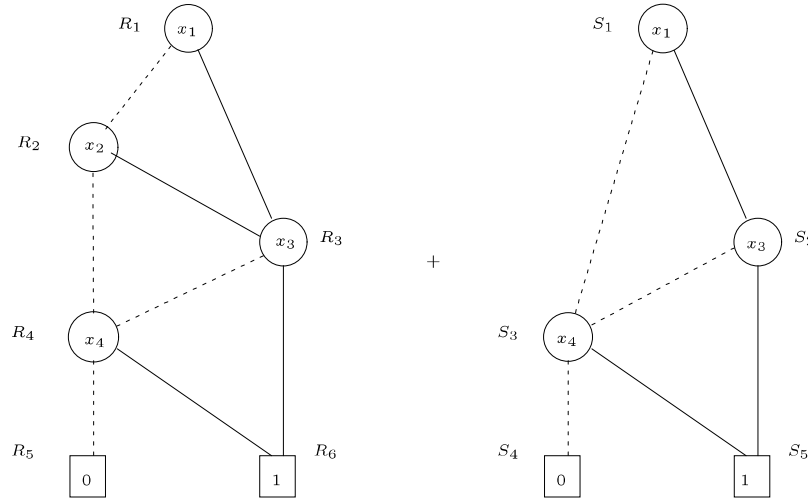


Figure 6.15. An example of two arguments for a call `apply (+,  $B_f, B_g$ )`.

of B_f and B_g downwards to construct nodes of the OBDD $B_{f \text{ op } g}$. Let r_f be the root node of B_f and r_g the root node of B_g .

1. If both r_f and r_g are terminal nodes with labels l_f and l_g , respectively (recall that terminal labels are either 0 or 1), then we compute the value $l_f \text{ op } l_g$ and let the resulting OBDD be B_0 if that value is 0 and B_1 otherwise.
2. In the remaining cases, at least one of the root nodes is a non-terminal. Suppose that both root nodes are x_i -nodes. Then we create an x_i -node n with a dashed line to `apply (op, lo( $r_f$ ), lo( $r_g$ ))` and a solid line to `apply (op, hi( $r_f$ ), hi( $r_g$ ))`, i.e. we call `apply` recursively on the basis of (6.2).
3. If r_f is an x_i -node, but r_g is a terminal node or an x_j -node with $j > i$, then we know that there is no x_i -node in B_g because the two OBDDs have a compatible ordering of boolean variables. Thus, g is independent of x_i ($g \equiv g[0/x_i] \equiv g[1/x_i]$). Therefore, we create an x_i -node n with a dashed line to `apply (op, lo( $r_f$ ),  $r_g$ )` and a solid line to `apply (op, hi( $r_f$ ),  $r_g$ )`.
4. The case in which r_g is a non-terminal, but r_f is a terminal or an x_j -node with $j > i$, is handled symmetrically to case 3.

The result of this procedure might not be reduced; therefore `apply` finishes by calling the function `reduce` on the OBDD it constructed. An example of `apply` (where `op` is `+`) can be seen in Figures 6.15–6.17. Figure 6.16 shows the recursive descent control structure of `apply` and Figure 6.17 shows the final result. In this example, the result of `apply (+,  $B_f, B_g$ )` is B_f .

Figure 6.16 shows that numerous calls to `apply` occur several times with the same arguments. Efficiency could be gained if these were evaluated only

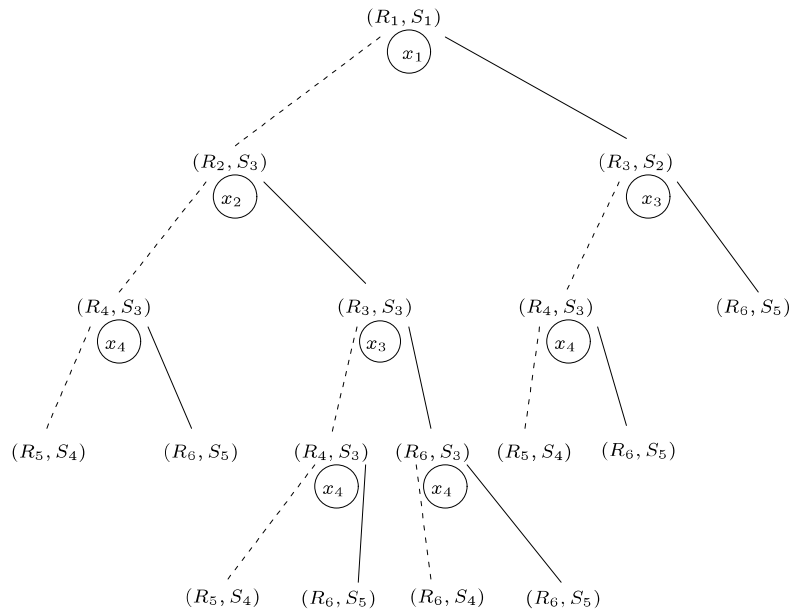


Figure 6.16. The recursive call structure of `apply` for the example in Figure 6.15 (without memoisation).

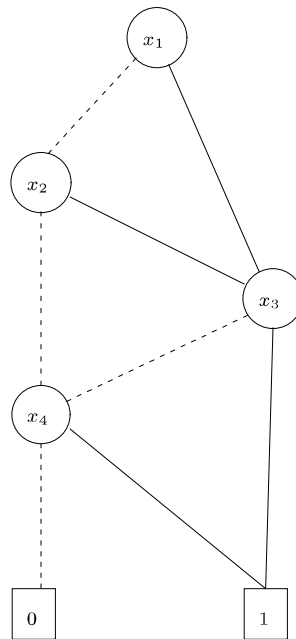


Figure 6.17. The result of `apply (+, Bf, Bg)`, where B_f and B_g are given in Figure 6.15.

the first time and the result remembered for future calls. This programming technique is known as memoisation. As well as being more efficient, it has the advantage that the resulting OBDD requires less reduction. (In this example, using memoisation eliminates the need for the final call to **reduce** altogether.) Without memoisation, **apply** is exponential in the size of its arguments, since each non-leaf call generates a further two calls. With memoisation, the number of calls to **apply** is bounded by $2 \cdot |B_f| \cdot |B_g|$, where $|B|$ is the size of the BDD. This is a worst-time complexity; the actual performance is often much better than this.

6.2.3 The algorithm **restrict**

Given an OBDD B_f representing a boolean formula f , we need an algorithm **restrict** such that the call **restrict**(0, x , B_f) computes the reduced OBDD representing $f[0/x]$ using the same variable ordering as B_f . The algorithm for **restrict**(0, x , B_f) works as follows. For each node n labelled with x , incoming edges are redirected to $\text{lo}(n)$ and n is removed. Then we call **reduce** on the resulting OBDD. The call **restrict**(1, x , B_f) proceeds similarly, only we now redirect incoming edges to $\text{hi}(n)$.

6.2.4 The algorithm **exists**

A boolean function can be thought of as putting a constraint on the values of its argument variables. For example, the function $x + (\bar{y} \cdot z)$ evaluates to 1 only if x is 1; or y is 0 and z is 1 – this is a constraint on x , y , and z .

It is useful to be able to express the relaxation of the constraint on a subset of the variables concerned. To allow this, we write $\exists x. f$ for the boolean function f with the constraint on x relaxed. Formally, $\exists x. f$ is defined as $f[0/x] + f[1/x]$; that is, $\exists x. f$ is true if f could be made true by putting x to 0 or to 1. Given that $\exists x. f \stackrel{\text{def}}{=} f[0/x] + f[1/x]$ the **exists** algorithm can be implemented in terms of the algorithms **apply** and **restrict** as

$$\mathbf{apply}(+, \mathbf{restrict}(0, x, B_f), \mathbf{restrict}(1, x, B_f)) . \quad (6.3)$$

Consider, for example, the OBDD B_f for the function $f \stackrel{\text{def}}{=} x_1 \cdot y_1 + x_2 \cdot y_2 + x_3 \cdot y_3$, shown in Figure 6.19. Figure 6.20 shows **restrict**(0, x_3 , B_f) and **restrict**(1, x_3 , B_f) and the result of applying $+$ to them. (In this case the **apply** function happens to return its second argument.)

We can improve the efficiency of this algorithm. Consider what happens during the **apply** stage of (6.3). In that case, the **apply** algorithm works on two BDDs which are identical all the way down to the level of the x -nodes;

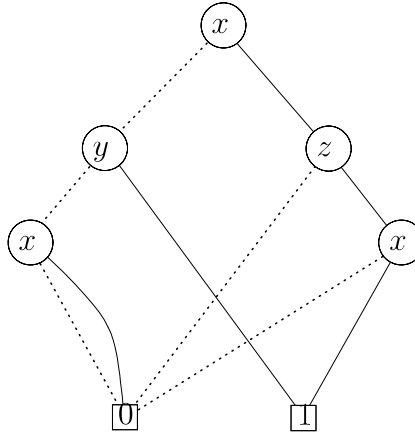


Figure 6.18. An example of a BDD which is not a read-1-BDD.

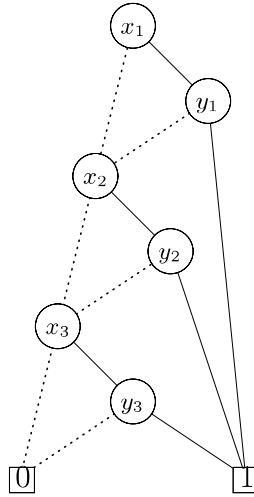


Figure 6.19. A BDD B_f to illustrate the exists algorithm.

therefore the returned BDD also has that structure down to the x -nodes. At the x -nodes, the two argument BDDs differ, so the **apply** algorithm will compute the apply of $+$ to these two subBDDs and return that as the subBDD of the result. This is illustrated in Figure 6.20. Therefore, we can compute the OBDD for $\exists x. f$ by taking the OBDD for f and replacing each node labelled with x by the result of calling **apply** on $+$ and its two branches.

This can easily be generalised to a sequence of **exists** operations. We write $\exists \hat{x}. f$ to mean $\exists x_1. \exists x_2. \dots \exists x_n. f$, where $\hat{x}$ denotes $(x_1, x_2, \dots, x_n)$.

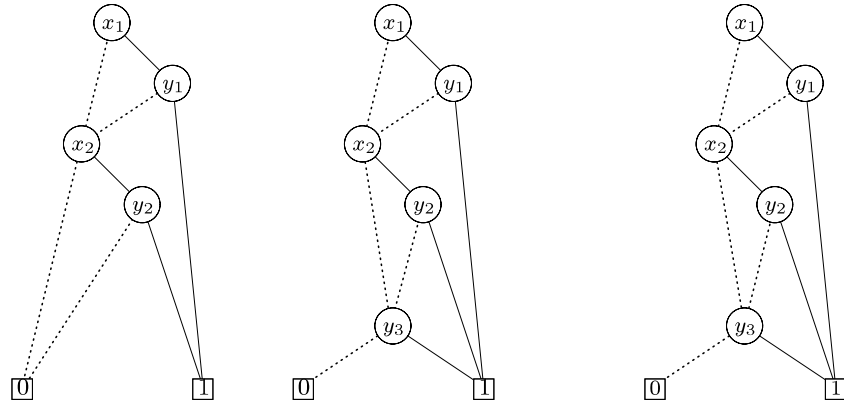


Figure 6.20. $\text{restrict}(0, x_3, B_f)$ and $\text{restrict}(1, x_3, B_f)$ and the result of applying $+$ to them.

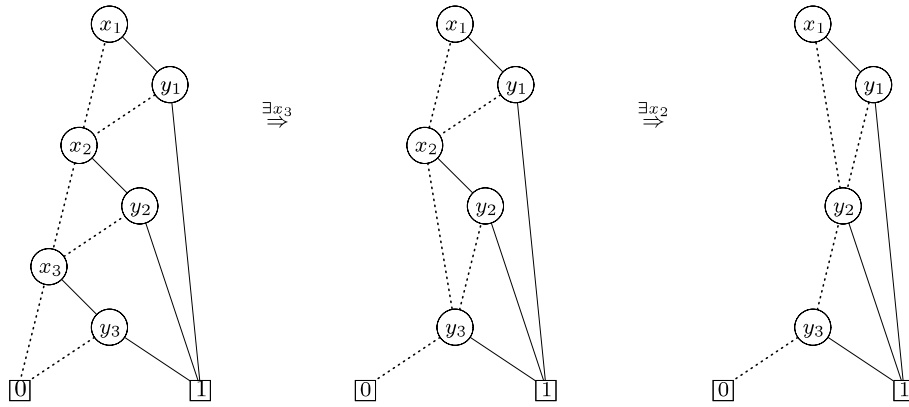


Figure 6.21. OBDDs for f , $\exists x_3.f$ and $\exists x_2.\exists x_3.f$.

The OBDD for this boolean function is obtained from the OBDD for f by replacing *every* node labelled with an x_i by the $+$ of its two branches.

Figure 6.21 shows the computation of $\exists x_3.f$ and $\exists x_2.\exists x_3.f$ (which is semantically equivalent to $x_1 \cdot y_1 + y_2 + y_3$) in this way.

The boolean quantifier $\forall$ is the dual of $\exists$:

$$\forall x.f \stackrel{\text{def}}{=} f[0/x] \cdot f[1/x]$$

asserting that f could be made false by putting x to 0 or to 1.

The translation of boolean formulas into OBDDs using the algorithms of this section is summarised in Figure 6.22.

Boolean formula f	Representing OBDD B_f
0	B_0 (Fig. 6.6)
1	B_1 (Fig. 6.6)
x	B_x (Fig. 6.6)
$\overline{f}$	swap the 0- and 1-nodes in B_f
$f + g$	apply $(+, B_f, B_g)$
$f \cdot g$	apply $(\cdot, B_f, B_g)$
$f \oplus g$	apply $(\oplus, B_f, B_g)$
$f[1/x]$	restrict $(1, x, B_f)$
$f[0/x]$	restrict $(0, x, B_f)$
$\exists x.f$	apply $(+, B_{f[0/x]}, B_{f[1/x]})$
$\forall x.f$	apply $(\cdot, B_{f[0/x]}, B_{f[1/x]})$

Figure 6.22. Translating boolean formulas f to OBDDs B_f , given a fixed, global ordering on boolean variables.

Algorithm	Input OBDD(s)	Output OBDD	Time-complexity
reduce	B	reduced B	$O(B \cdot \log B)$
apply	B_f, B_g (reduced)	$B_{f \text{ op } g}$ (reduced)	$O(B_f \cdot B_g)$
restrict	B_f (reduced)	$B_{f[0/x]}$ or $B_{f[1/x]}$ (reduced)	$O(B_f \cdot \log B_f)$
$\exists$	B_f (reduced)	$B_{\exists x_1. \exists x_2. \dots \exists x_n. f}$ (reduced)	NP-complete

Figure 6.23. Upper bounds in terms of the input OBDD(s) for the worst-case running times of our algorithms needed in our implementation of boolean formulas.

6.2.5 Assessment of OBDDs

Time complexities for computing OBDDs We can measure the complexity of the algorithms of the preceding section by giving upper bounds for the running time in terms of the sizes of the input OBDDs. The table in Figure 6.23 summarises these upper bounds (some of those upper bounds may require more sophisticated versions of the algorithms than the versions presented in this chapter). All the operations except nested boolean quantification are practically efficient in the size of the participating OBDDs. Thus, modelling very large systems with this approach will work if the OBDDs

which represent the systems don't grow too large too fast. If we can somehow control the size of OBDDs, e.g. by using good heuristics for the choice of variable ordering, then these operations are computationally feasible. It has already been shown that OBDDs modelling certain classes of systems and networks don't grow excessively.

The expensive computational operations are the nested boolean quantifications $\exists z_1 \dots \exists z_n.f$ and $\forall z_1 \dots \forall z_n.f$. By exercise 1 on page 406, the computation of the OBDD for $\exists z_1 \dots \exists z_n.f$, given the OBDD for f , is an NP-complete problem²; thus, it is unlikely that there exists an algorithm with a feasible worst-time complexity. This is not to say that boolean functions modelling practical systems may not have efficient nested boolean quantifications. The performance of our algorithms can be improved by using further optimisation techniques, such as parallelisation.

Note that the operations **apply**, **restrict**, etc. are only efficient in the size of the input OBDDs. So if a function f does not have a compact representation as an OBDD, then computing with its OBDD will not be efficient. There are such nasty functions; indeed, one of them is *integer multiplication*. Let $b_{n-1}b_{n-2} \dots b_0$ and $a_{n-1}a_{n-2} \dots a_0$ be two n -bit integers, where b_{n-1} and a_{n-1} are the most significant bits and b_0 and a_0 are the least significant bits. The multiplication of these two integers results in a $2n$ -bit integer. Thus, we may think of multiplication as $2n$ many boolean functions f_i in $2n$ variables (n bits for input b and n bits for input a), where f_i denotes the i th output bit of the multiplication. The following negative result, due to R. E. Bryant, shows that OBDDs cannot be used for implementing integer multiplication.

Theorem 6.11 Any OBDD representation of f_{n-1} has at least a number of vertices proportional to 1.09^n , i.e. its size is exponential in n .

Extensions and variations of OBDDs There are many variations and extensions to the OBDD data structure. Many of them can implement certain operations more efficiently than their OBDD counterparts, but it seems that none of them perform as well as OBDDs overall. In particular, one feature which many of the variations lack is the canonical form; therefore they lack an efficient algorithm for deciding when two objects denote the same boolean function.

One kind of variation allows non-terminal nodes to be labelled with binary operators as well as boolean variables. *Parity OBDDs* are like OBDDs in that there is an ordering on variables and every variable may occur at

² Another NP-complete problem is to decide the satisfiability of formulas of propositional logic.

1 at $\hat{x}$ if there is a path that realises the $f \text{ U } g$ pattern:

$$\text{checkEU}(f, g) \stackrel{\text{def}}{=} \mu Y. g + (f \cdot \text{checkEX}(Y)). \quad (6.26)$$

With this in place, we can code $f^{\text{ECG}\phi}$ quite easily:

$$f^{\text{ECG}\phi} \stackrel{\text{def}}{=} \nu Z. f^\phi \cdot \prod_{i=1}^k \text{checkEX}(\text{checkEU}(f^\phi, Z \cdot f^{\psi_i}) \cdot \text{fair}). \quad (6.27)$$

Note that this coding has a least fixed point (**checkEU**) in the body of a greatest fixed point. This is computationally rather involved since the call of **checkEU** contains Z , the recursion variable of the outer greatest fixed point, as a free variable; thus these recursions are nested and inter-dependent; the recursions ‘alternate.’ Observe how this coding operates: to have a fair path from $\hat{x}$ on which ϕ holds globally, we need ϕ to hold at $\hat{x}$; and for all fairness constraints ψ_i there has to be a next state $\hat{x}'$, where the whole property is true again (enforced by the free Z) and each fairness constraint is realised eventually on that path. The recursion in Z constantly reiterates this reasoning, so if this function computes 1, then there is a path on which ϕ holds globally and where each ψ_i is true infinitely often.

6.5 Exercises

Exercises 6.1

1. Write down the truth tables for the boolean formulas in Example 6.2 on page 359. In your table, you may use 0 and 1, or F and T, whatever you prefer. What truth value does the boolean formula of item (4) on page 359 compute?
2. $\oplus$ is the exclusive-or: $x \oplus y \stackrel{\text{def}}{=} 1$ if the values of x and y are different; otherwise, $x \oplus y \stackrel{\text{def}}{=} 0$. Express this in propositional logic, i.e. find a formula ϕ having the same truth table as $\oplus$.
- * 3. Write down a boolean formula $f(x, y)$ in terms of $\cdot$, $+$, $\neg$, 0 and 1, such that f has the same truth table as $p \rightarrow q$.
4. Write down a BNF for the syntax of boolean formulas based on the operations in Definition 6.1.

Exercises 6.2

- * 1. Suppose we swap all dashed and solid lines in the binary decision tree of Figure 6.2. Write out the truth table of the resulting binary decision tree and find a formula for it.

- * 2. Consider the following truth table:

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	F
T	F	F	F
F	T	T	T
F	T	F	F
F	F	T	T
F	F	F	F

Write down a binary decision tree which represents the boolean function specified in this truth table.

3. Construct a binary decision tree for the boolean function specified in Figure 6.2, but now the root should be a y -node and its two successors should be x -nodes.
4. Consider the following boolean function given by its truth table:

x	y	z	$f(x, y, z)$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	0
0	0	0	1

- (a) Construct a binary decision tree for $f(x, y, z)$ such that the root is an x -node followed by y - and then z -nodes.
- (b) Construct another binary decision tree for $f(x, y, z)$, but now let its root be a z -node followed by y - and then x -nodes.
5. Let T be a binary decision tree for a boolean function $f(x_1, x_2, \dots, x_n)$ of n boolean variables. Suppose that every variable occurs exactly once as one travels down on any path of the tree T . Use mathematical induction to show that T has $2^{n+1} - 1$ nodes.

Exercises 6.3

- * 1. Explain why all reductions C1–C3 (page 363) on a BDD B result in BDDs which still represent the same function as B .
2. Consider the BDD in Figure 6.7.
- * (a) Specify the truth table for the boolean function $f(x, y, z)$ represented by this BDD.

- (b) Find a BDD for that function which does not have multiple occurrences of variables along any path.
- 3. Let f be the function represented by the BDD of Figure 6.3(b). Using also the BDDs B_0 , B_1 and B_x illustrated in Figure 6.6, find BDDs representing
 - (a) $f \cdot x$
 - (b) $x + f$
 - (c) $\overline{f \cdot 0}$
 - (d) $f \cdot 1$.

Exercises 6.4

1. Figure 6.9 (page 367) shows a BDD with ordering $[x, y, z]$.
 - * (a) Find an equivalent reduced BDD with ordering $[z, y, x]$. (Hint: find first the decision tree with the ordering $[z, y, x]$, and then reduce it using C1–C3.)
 - (b) Carry out the same construction process for the variable ordering $[y, z, x]$. Does the reduced BDD have more or fewer nodes than the ones for the orderings $[x, y, z]$ and $[z, y, x]$?
2. Consider the BDDs in Figures 6.4–6.10. Determine which of them are OBDDs. If you find an OBDD, you need to specify a list of its boolean variables without double occurrences which demonstrates that ordering.
3. Consider the following boolean formulas. Compute their unique reduced OBDDs with respect to the ordering $[x, y, z]$. It is advisable to first compute a binary decision tree and then to perform the removal of redundancies.
 - (a) $f(x, y) \stackrel{\text{def}}{=} x \cdot y$
 - * (b) $f(x, y) \stackrel{\text{def}}{=} x + y$
 - (c) $f(x, y) \stackrel{\text{def}}{=} x \oplus y$
 - * (d) $f(x, y, z) \stackrel{\text{def}}{=} (x \oplus y) \cdot (\overline{x} + z)$.
4. Recall the derived connective $\phi \leftrightarrow \psi$ from Chapter 1 saying that for all valuations ϕ is true if, and only if, ψ is true.
 - (a) Define this operator for boolean formulas using the basic operations $\cdot$, $+$, $\oplus$ and $\neg$ from Definition 6.1.
 - (b) Draw a reduced OBDD for the formula $g(x, y) \stackrel{\text{def}}{=} x \leftrightarrow y$ using the ordering $[y, x]$.
5. Consider the even parity function introduced at the end of the last section.
 - (a) Define the odd parity function $f_{\text{odd}}(x_1, x_2, \dots, x_n)$.
 - (b) Draw an OBDD for the odd parity function for $n = 5$ and the ordering $[x_3, x_5, x_1, x_4, x_2]$. Would the overall structure of this OBDD change if you changed the ordering?
 - (c) Show that $f_{\text{even}}(x_1, x_2, \dots, x_n)$ and $\overline{f_{\text{odd}}(x_1, x_2, \dots, x_n)}$ denote the same boolean function.
6. Use Theorem 6.7 (page 368) to show that, if the reductions C1–C3 are applied until no more reduction is possible, the result is independent of the order in which they were applied.

Exercises 6.5

1. Given the boolean formula $f(x_1, x_2, x_3) \stackrel{\text{def}}{=} x_1 \cdot (x_2 + \overline{x_3})$, compute its reduced OBDD for the following orderings:
 - (a) $[x_1, x_2, x_3]$
 - (b) $[x_3, x_1, x_2]$
 - (c) $[x_3, x_2, x_1]$.
2. Compute the reduced OBDD for $f(x, y, z) = x \cdot (z + \overline{z}) + \overline{y} \cdot \overline{x}$ in any ordering you like. Is there a z -node in that reduced OBDD?
3. Consider the boolean formula $f(x, y, z) \stackrel{\text{def}}{=} (\overline{x} + y + \overline{z}) \cdot (x + \overline{y} + z) \cdot (x + y)$. For the variable orderings below, compute the (unique) reduced OBDD B_f of f with respect to that ordering. It is best to write down the binary decision tree for that ordering and then to apply all possible reductions.
 - (a) $[x, y, z]$.
 - (b) $[y, x, z]$.
 - (c) $[z, x, y]$.
 - (d) Find an ordering of variables for which the resulting reduced OBDD B_f has a minimal number of edges; i.e. there is no ordering for which the corresponding B_f has fewer edges. (How many possible orderings for x, y and z are there?)
4. Given the truth table

x	y	z	$f(x, y, z)$
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	1

compute the reduced OBDD with respect to the following ordering of variables:

- (a) $[x, y, z]$
 - (b) $[z, y, x]$
 - (c) $[y, z, x]$
 - (d) $[x, z, y]$.
5. Given the ordering $[p, q, r]$, compute the reduced BDDs for $p \wedge (q \vee r)$ and $(p \wedge q) \vee (p \wedge r)$ and explain why they are identical.
 - * 6. Consider the BDD in Figure 6.11 (page 370).
 - (a) Construct its truth table.
 - (b) Compute its conjunctive normal form.
 - (c) Compare the length of that normal form with the size of the BDD. What is your assessment?

Exercises 6.6

1. Perform the execution of **reduce** on the following OBDDs:

(a) The binary decision tree for

- i. $x \oplus y$
- ii. $x \cdot y$
- iii. $x + y$
- iv. $x \leftrightarrow y$.

(b) The OBDD in Figure 6.2 (page 361).

* (c) The OBDD in Figure 6.4 (page 363).

Exercises 6.7

1. Recall the Shannon expansion in (6.1) on page 374. Suppose that x does not occur in f at all. Why does (6.1) still hold?
2. Let $f(x, y, z) \stackrel{\text{def}}{=} y + \bar{z} \cdot x + z \cdot \bar{y} + y \cdot x$ be a boolean formula. Compute f 's Shannon expansion with respect to
 - (a) x
 - (b) y
 - (c) z .

3. Show that boolean formulas f and g are semantically equivalent if, and only if, the boolean formula $(\bar{f} + g) \cdot (f + \bar{g})$ computes 1 for all possible assignments of 0s and 1s to their variables.

4. We may use the Shannon expansion to define formally how BDDs determine boolean functions. Let B be a BDD. It is intuitively clear that B determines a unique boolean function. Formally, we compute a function f_n inductively (bottom-up) for all nodes n of B :

- If n is a terminal node labelled 0, then f_n is the constant 0 function.
- Dually, if n is a terminal 1-node, then f_n is the constant 1 function.
- If n is a non-terminal node labelled x , then we already have defined the boolean functions $f_{\text{lo}(n)}$ and $f_{\text{hi}(n)}$ and set f_n to be $\bar{x} \cdot f_{\text{lo}(n)} + x \cdot f_{\text{hi}(n)}$.

If i is the initial node of B , then f_i is the boolean function represented by B . Observe that we could apply this definition as a symbolic evaluation of B resulting in a boolean formula. For example, the BDD of Figure 6.3(b) renders $\bar{x} \cdot (\bar{y} \cdot 1 + y \cdot 0) + x \cdot 0$. Compute the boolean formulas obtained in this way for the following BDDs:

- (a) the BDD in Figure 6.5(b) (page 364)
- (b) the BDDs in Figure 6.6 (page 365)
- (c) the BDD in Figure 6.11 (page 370).

* 5. Consider a ternary (= takes three arguments) boolean connective $f \rightarrow (g, h)$ which is equivalent to g when f is true; otherwise, it is equivalent to h .

- (a) Define this connective using any of the operators $+$, $\cdot$, $\oplus$ or $\bar{}$.
- (b) Recall exercise 4. Use the ternary operator above to write f_n as an expression of $f_{\text{lo}(n)}$, $f_{\text{hi}(n)}$ and its label x .

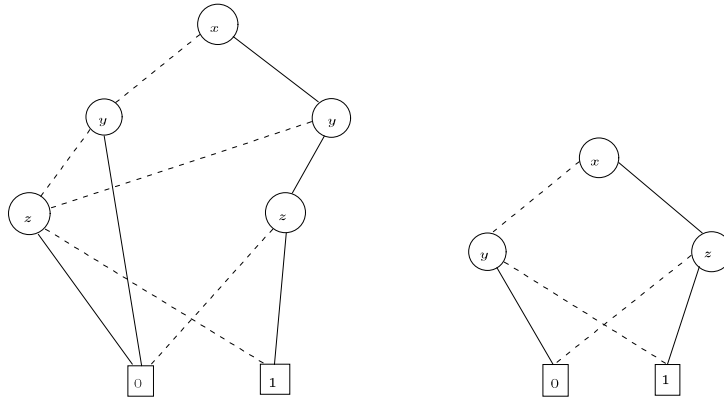


Figure 6.30. The reduced OBDDs B_f and B_g (see exercises).

- (c) Use mathematical induction (on what?) to prove that, if the root of f_n is an x -node, then f_n is independent of any y which comes before x in an assumed variable ordering.
6. Explain why **apply** (op, B_f, B_g), where B_f and B_g have compatible ordering, produces an OBDD with an ordering compatible with that of B_f and B_g .
 7. Explain why the four cases of the control structure for **apply** are exhaustive, i.e. there are no other possible cases in its execution.
 8. Consider the reduced OBDDs B_f and B_g in Figure 6.30. Recall that, in order to compute the reduced OBDD for $f \text{ op } g$, you need to
 - construct the tree showing the recursive descent of **apply** (op, B_f, B_g) as done in Figure 6.16;
 - use that tree to simulate **apply** (op, B_f, B_g); and
 - reduce, if necessary, the resulting OBDD.
 Perform these steps on the OBDDs of Figure 6.30 for the operation ‘op’ being
 - (a) $+$
 - (b) $\oplus$
 - (c) $\cdot$
 9. Let B_f be the OBDD in Figure 6.11 (page 370). Compute **apply** ($\oplus, B_f, B_1$) and reduce the resulting OBDD. If you did everything correctly, then this OBDD should be isomorphic to the one obtained from swapping 0- and 1-nodes in Figure 6.11.
 - * 10. Consider the OBDD B_c in Figure 6.31 which represents the ‘don’t care’ conditions for comparing the boolean functions f and g represented in Figure 6.30. This means that we want to compare whether f and g are equal for all values of variables except those for which c is true (i.e. we ‘don’t care’ when c is true).
 - (a) Show that the boolean formula $(\bar{f} \oplus g) + c$ is valid (always computes 1) if, and only if, f and g are equivalent on all values for which c evaluates to 0.

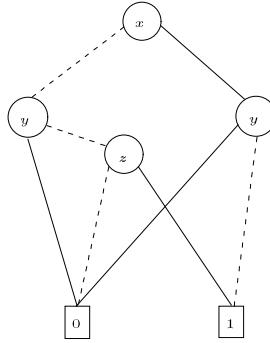


Figure 6.31. The reduced OBDD B_c representing the ‘don’t care’ conditions for the equivalence test of the OBDDs in Figure 6.30.

- (b) Proceed in three steps as in exercise 8 on page 403 to compute the reduced OBDD for $(\bar{f} \oplus g) + c$ from the OBDDs for f , g and c . Which call to **apply** needs to be first?
- 11. We say that $v \in \{0, 1\}$ is a (left)-controlling value for the operation op , if either $v \text{ op } x = 1$ or $v \text{ op } x = 0$ for all values of x . We say that v is a controlling value if it is a left- and right-controlling value.
 - (a) Define the notion of a right-controlling value.
 - (b) Give examples of operations with controlling values.
 - (c) Describe informally how **apply** can be optimised when op has a controlling value.
 - (d) Could one still do some optimisation if op had only a left- or right-controlling value?
- 12. We showed that the worst-time complexity of **apply** is $O(|B_f| \cdot |B_g|)$. Show that this upper bound is hard, i.e. it cannot be improved:
 - (a) Consider the functions $f(x_1, x_2, \dots, x_{2n+2m}) \stackrel{\text{def}}{=} x_1 \cdot x_{n+m+1} + \dots + x_n \cdot x_{2n+m}$ and $g(x_1, x_2, \dots, x_{2n+2m}) \stackrel{\text{def}}{=} x_{n+1} \cdot x_{2n+m+1} + \dots + x_{n+m} \cdot x_{2n+2m}$ which are in sum-of-product form. Compute the sum-of-product form of $f + g$.
 - (b) Choose the ordering $[x_1, x_2, \dots, x_{2n+2m}]$ and argue that the OBDDs B_f and B_g have 2^{n+1} and 2^{m+1} edges, respectively.
 - (c) Use the result from part (a) to conclude that B_{f+g} has 2^{n+m+1} edges, i.e. $0.5 \cdot |B_f| \cdot |B_g|$.

Exercises 6.8

- 1. Let f be the reduced OBDD represented in Figure 6.5(b) (page 364). Compute the reduced OBDD for the restrictions:
 - (a) $f[0/x]$
 - * (b) $f[1/x]$

- (c) $f[1/y]$
 - * (d) $f[0/z]$.
- * 2. Suppose that we intend to modify the algorithm **restrict** so that it is capable of computing reduced OBDDs for a general composition $f[g/x]$.
- (a) Generalise Equation (6.1) to reflect the intuitive meaning of the operation $[g/x]$.
 - (b) What fact about OBDDs causes problems for computing this composition directly?
 - (c) How can we compute this composition given the algorithms discussed so far?
3. We define read-1-BDDs as BDDs B where each boolean variable occurs at most once on any evaluation path of B . In particular, read-1-BDDs need not possess an ordering on their boolean variables. Clearly, every OBDD is a read-1-BDD; but not every read-1-BDD is an OBDD (see Figure 6.10). In Figure 6.18 we see a BDD which is not a read-1-BDD; the path for $(x, y, z) \Rightarrow (1, 0, 1)$ ‘reads’ the value of x twice.
- Critically assess the implementation of boolean formulas via OBDDs to see which implementation details could be carried out for read-1-BDDs as well. Which implementation aspects would be problematic?
4. (For those who have had a course on finite automata.) Every boolean function f in n arguments can be viewed as a subset L_f of $\{0, 1\}^n$; defined to be the set of all those bit vectors $(v_1, v_2, \dots, v_n)$ for which f computes 1. Since this is a finite set, L_f is a regular language and has therefore a deterministic finite automaton with a minimal number of states which accepts L_f . Can you match some of the OBDD operations with those known for finite automata? How close is the correspondence? (You may have to consider non-reduced OBDDs.)
5. (a) Show that every boolean function in n arguments can be represented as a boolean formula of the grammar
- $$f ::= 0 \mid x \mid \overline{f} \mid f_1 + f_2.$$
- (b) Why does this also imply that every such function can be represented by a reduced OBDD in any variable ordering?
6. Use mathematical induction on n to prove that there are exactly $2^{(2^n)}$ many different boolean functions in n arguments.

Exercises 6.9

1. Use the **exists** algorithm to compute the OBDDs for
 - (a) $\exists x_3.f$, given the OBDD for f in Figure 6.11 (page 370)
 - (b) $\forall y.g$, given the OBDD for g in Figure 6.9 (page 367)
 - (c) $\exists x_2.\exists x_3.x_1 \cdot y_1 + x_2 \cdot y_2 + x_3 \cdot y_3$.
2. Let f be a boolean function depending on n variables.
 - (a) Show:

- i. The formula $\exists x.f$ depends on all those variables that f depends upon, except x .
 - ii. If f computes to 1 with respect to a valuation ρ , then $\exists x.f$ computes 1 with respect to the same valuation.
 - iii. If $\exists x.f$ computes to 1 with respect to a valuation ρ , then there is a valuation ρ' for f which agrees with ρ for all variables other than x such that f computes to 1 under ρ' .
- (b) Can the statements above be shown for the function value 0?
3. Let ϕ be a boolean formula.
- * (a) Show that ϕ is satisfiable if, and only if, $\exists x.\phi$ is satisfiable.
 - (b) Show that ϕ is valid if, and only if, $\forall x.\phi$ is valid.
 - (c) Generalise the two facts above to nested quantifications $\exists \hat{x}$ and $\forall \hat{x}$. (Use induction on the number of quantified variables.)
4. Show that $\forall \hat{x}.f$ and $\exists \hat{x}.\bar{f}$ are semantically equivalent. Use induction on the number of arguments in the vector $\hat{x}$.
-

Exercises 6.10

(For those who know about complexity classes.)

1. Show that 3SAT can be reduced to nested existential boolean quantification. Given an instance of 3SAT, we may think of it as a boolean formula f in product-of-sums form $g_1 \cdot g_2 \cdots g_n$, where each g_i is of the form $(l_1 + l_2 + l_3)$ with each l_j being a boolean variable or its complementation. For example, f could be $(x + \bar{y} + z) \cdot (x_5 + x + \bar{x}_7) \cdot (\bar{x}_2 + z + x) \cdot (x_4 + \bar{x}_2 + \bar{x}_4)$.
 - (a) Show that you can represent each function g_i with an OBDD of no more than three non-terminals, independently of the chosen ordering.
 - (b) Introduce n new boolean variables $z_1, z_2, \dots, z_n$. We write $\sum_{1 \leq i \leq n} f_i$ for the expression $f_1 + f_2 + \cdots + f_n$ and $\prod_{1 \leq i \leq n} f_i$ for $f_1 \cdot f_2 \cdots f_n$. Consider the boolean formula h , defined as

$$\sum_{1 \leq i \leq n} \left(\bar{g}_i \cdot z_i \cdot \prod_{1 \leq j < i} \bar{z}_j \right). \quad (6.28)$$

Choose any ordering of variables whose list begins as in $[z_1, z_2, \dots, z_n, \dots]$.

Draw the OBDD for h (draw only the root nodes for $\bar{g}_i$).

- (c) Argue that the OBDD above has at most $4n$ non-terminal nodes.
 - (d) Show that f is satisfiable if, and only if, the OBDD for $\exists z_1. \exists z_2. \dots \exists z_n. h$ is not equal to B_1 .
 - (e) Explain why the last item shows a reduction of 3SAT to nested existential quantification.
2. Show that the problem of finding an optimal ordering for representing boolean functions as OBDDs is in coNP.

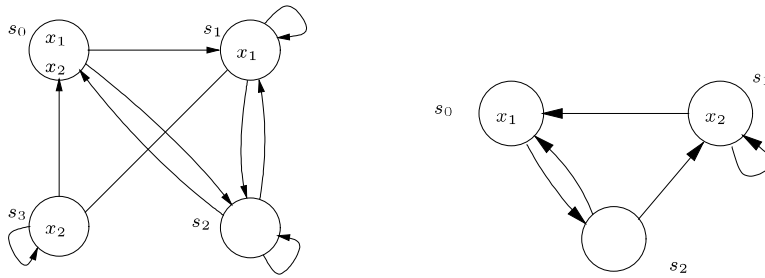


Figure 6.32. (a) A CTL model with four states. (b) A CTL model with three states.

3. Recall that $\exists x.f$ is defined as $f[1/x] + f[0/x]$. Since we have efficient algorithms for restriction and $+$, we obtain hereby an efficient algorithm for $\exists z_1 \dots \exists z_n.f$. Thus, P equals NP! What is wrong with this argument?

Exercises 6.11

- * 1. Consider the CTL model in Figure 6.32(a). Using the ordering $[x_1, x_2]$, draw the OBDD for the subsets $\{s_0, s_1\}$ and $\{s_0, s_2\}$.
2. Consider the CTL model in Figure 6.32(b). Because the number of states is not an exact power of 2, there are more than one OBDDs representing any given set of states. Using again the ordering $[x_1, x_2]$, draw all possible OBDDs for the subsets $\{s_0, s_1\}$ and $\{s_0, s_2\}$.

Exercises 6.12

1. Consider the CTL model in Figure 6.32(a).
- (a) Work out the truth table for the transition relation, ordering the columns $[x_1, x'_1, x_2, x'_2]$. There should be as many 1s in the final column as there are arrows in the transition relation. There is no freedom in the representation in this case, since the number of states is an exact power of 2.
 - (b) Draw the OBDD for this transition relation, using the variable ordering $[x_1, x'_1, x_2, x'_2]$.
2. Apply the algorithm of Section 3.6.1, but now interpreted over OBDDs in the ordering $[x_1, x_2]$, to compute the set of states of the CTL model in Figure 6.32(b) which satisfy
- (a) $\text{AG}(x_1 \vee \neg x_2)$
 - (b) $\text{E}[x_2 \text{ U } x_1]$.
- Show the OBDDs which are computed along the way.
3. Explain why $\text{exists}(\hat{x}', \text{apply}(\cdot, B_{\neg}, B_{X'}))$ faithfully implements the meaning of $\text{pre}_{\exists}(X)$.